



Wstęp	2
Front-end	3
HTML	4
CSS.....	4
JavaScript.....	5
index.js	5
modify-table.js	8
comparison.js	9
Back-end.....	11
PHP	11
config.php.....	11
metadata_form.php	11
random_records.php	12
record_deletion.php.....	13
record_form.php.....	13
table_deletion.php.....	14
api_importing.php.....	14

Wstęp

TablixVS to interaktywna aplikacja umożliwiająca użytkownikom wybór tabel i porównywanie wartości ich elementów w celu ustalenia, który z nich jest większy. Aplikacja obsługuje zarówno import tabel z API Spotify, jak i możliwość ich ręcznego dodawania. Użytkownicy mają także możliwość modyfikowania tabel, w tym dodawania lub usuwania rekordów oraz usuwania całych tabel. Interfejs użytkownika jest zaprojektowany z myślą o intuicyjnej obsłudze.

Funkcjonalności:

- Wybór tabeli z dostępnej listy.
- Import tabel z API Spotify.
- Dodawanie własnych tabel manualnie.
- Modyfikowanie tabel – dodawanie lub usuwanie rekordów.
- Usuwanie tabel.
- Dynamiczne porównywanie wartości elementów w tabeli.
- Intuicyjny i responsywny interfejs użytkownika.

Technologie:

- Frontend: HTML, CSS, JavaScript
- Backend: PHP
- Baza danych: MySQL

Front-end

Tworzenie

Twórz własne tabele na dowolny temat lub importuj swoje ukochane playlisty Spotify - wybór jest Twój!



Playlista



Tabela



Dodaj

Twoja tabela

Nazwa	Wartość
Rekord 1	100
Rekord 2	50
rekord 3	25
rekord 4	10

Link do playlisty

identyfikator

identyfikator

Przełącz dane z playlisty Wyczyść

Modyfikowanie

Dodawaj i usuwaj rekordy z tablic bez większego wysiłku!







Tutorial Importu

Nazwa	Wartość
Rekord 1	100
Rekord 2	50
rekord 3	25
rekord 4	10

Dodaj

HTML

Podstawowy układ strony jest stworzony wykorzystując HTML.

```
Podstrony > info.html > nml > body.grid > footer > div.footerSection > a.footerElement.footerLink
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <link rel="stylesheet" href="../Style/style.css">
7   <link rel="preconnect" href="https://fonts.googleapis.com">
8   <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
9   <link href="https://fonts.googleapis.com/css2?family=Montserrat:ital,wght@0,100..900;1,100..900&display=swap" rel="stylesheet">
10  <title>Info</title>
11 </head>
12 <body class="grid">
13   <div class="returnButton top-left">
14     <a href=".." />  </a>
15   </div>
16   <header>
17     
18   </header>
19   <main>
20     <div class="infoLayout sectionSize50">
21       <div class="infoSection">
22         <h2>Tworzenie</h2>
23         <p>Wdróż własne tabele na dowolny temat lub importuj swoje ulubione playlisty Spotify - wybór jest Twój!</p>
24       </div>
25       <div class="infoSection horizontal doubleWidth">
26         <div class="Option Spotify disabledAction">
27           <svg class="optionCover" id="optionCover">
28             <use href=".." />
29           </svg>
30           <p id="optionDescription" class="optionDescription">Playlist</p>
31         </div>
32         <div class="Option Custom disabledAction">
33           <svg class="optionCover" id="optionCover">
34             <use href=".." />
35           </svg>
36           <p id="optionDescription" class="optionDescription">Tabela</p>
37         </div>
38         <div class="Option add disabledAction">
39           <svg class="optionCover" id="optionCover">
40             <use href=".." />
41           </svg>
42           <p id="optionDescription" class="optionDescription">Dodaj</p>
43         </div>
44       </div>
45     </div>
46   </main>
47 </body>
48 </html>
```

CSS

Strona jest oparta na jednym pliku CSS zawierającym wiele przydatnych klas, używanych m.in. do stylizowania czy rozmieszczania elementów na stronie. Nazwy klas są zgodne z przeznaczeniem lub funkcją klasy – np. Klasa “grid” dodaje kratkę do tła, za to klasa “noPadding” usuwa padding w przypadku bardziej rozwiniętego elementu, który chcemy zmodyfikować. Pozwala to na łatwiejszy odczyt.

```
39 .optionField{
40   display: grid;
41   grid-template-columns: repeat(5, 1fr);
42   column-gap: 4%;
43   height: auto;
44   margin: 0 auto;
45   width: 60%;
46   justify-content: center;
47   position: relative;
48   grid-auto-flow: row;
49 }
50 .Option{
51   aspect-ratio: 12/16;
52   height: 380px;
53   margin-bottom: 40px;
54   border-radius: 12px;
55   display: flex;
56   flex-direction: column;
57   align-items: center;
58   justify-content: center;
59   padding: 8px;
60   cursor: pointer;
61   transition: transform 0.3s ease;
62   box-shadow: 0px 5px 20px 0px rgba(0,0,0,0.6) inset, 0px 7.5px 20px 0px rgba(0,0,0,0.6);
63   position: relative;
64 }
65 .Option:hover{
66   transform: translateY(-5%);
67 }
68 .optionCover{
69   width: 75%;
70   aspect-ratio: 1/1;
71 }
72 .Custom{
73   background-color: var(--primary-color);
74 }
75 .Spotify{
76   background-color: var(--secondary-color);
77 }
78 .add{
79   background-color: var(--neutral-color);
80 }
81 /* ===== */
```

Wykorzystywane kolory są zapisane w zmiennych na początku pliku.

```
:root{
  --bg-color: #fafffd;
  --text-color: #0d1821;
  --text-color-alt: #fafffd;
  --primary-color: #6c91c1;
  --primary-color-alt: #5a7ca9;
  --secondary-color: #1db954;
  --neutral-color: #b8b8b8;
}
body{
```

JavaScript

Jeden plik JS odpowiada stronie, na której wykonuje zadania. W nim zawarte są odpowiednie skrypty.

index.js

Funkcja optionMenu obsługuje otwieranie i zarządzanie stylem i zawartością pojawiającego się po wybraniu opcji menu. W razie wybrania opcji, która jest odpowiedzialna za dodawanie tabel, otwiera się inne menu.

```
function optionMenu() {
  const clickCard = event.currentTarget;
  const selectedPage = document.getElementById('selectedPage');
  const tableActionMenu = document.getElementById('tableActionMenu');
  const tableCreateMenu = document.getElementById('tableCreateMenu');

  selectedPage.style.display = 'flex';
  document.body.style.overflow = 'hidden';

  if (clickCard.classList.contains('add')) {
    tableActionMenu.style.display = 'none';
    tableCreateMenu.style.display = 'flex';

    const primaryButton = tableActionMenu.querySelector('primaryButton');
    const cardStyle = window.getComputedStyle(clickCard);
    const backgroundColor = cardStyle.backgroundColor;
    primaryButton.style.color = backgroundColor;
  } else {
    const cardTitleElement = clickCard.querySelector('optionDescription');
    const cardContentElement = clickCard.querySelector('optionCover-url');

    const cardTitle = cardTitleElement.innerText;
    const cardIcon = cardContentElement.innerHTML;

    const primaryButton = tableActionMenu.querySelector('primaryButton');
    const actionMenuTitle = document.getElementById('actionMenuTitle');
    const actionMenuIcon = document.querySelector('tableActionMenu url');
    const cardStyle = window.getComputedStyle(clickCard);
    const backgroundColor = cardStyle.backgroundColor;

    actionMenuTitle.innerText = cardTitle;
    actionMenuIcon.innerHTML = cardIcon;
    tableActionMenu.style.backgroundColor = backgroundColor;
    primaryButton.style.color = backgroundColor;

    tableActionMenu.style.display = 'flex';
    tableCreateMenu.style.display = 'none';
  }
}
```

Obsługiwanie przycisków, aby odpowiednio przekierować użytkownika:

```
if (primaryButton) {
  primaryButton.onclick = function() {
    const tableName = document.getElementById('actionMenuTitle').innerText;
    const url = `../Podstrony/comparison.php?table_name=${tableName}`;
    window.location.href = url;
  };
}

const secondaryButton = document.querySelector('.secondaryButton');
if (secondaryButton) {
  secondaryButton.onclick = function() {
    const tableName = document.getElementById('actionMenuTitle').innerText;
    const url = `../Podstrony/modify-table.php?table_name=${tableName}`;
    window.location.href = url;
  };
}
```

Listener dla optionMenu, funkcja closeOptionMenu która zamyka menu, listener dla closeOptionMenu w przycisku exit

```

document.querySelectorAll('.Option').forEach(card => {
    card.addEventListener('click', optionMenu);
});

function closeOptionMenu(){
    const selectedPopUp = document.querySelector('.selectedPopUp');
    const tableActionMenu = document.getElementById('tableActionMenu');
    const tableCreationMenu = document.getElementById('tableCreationMenu');

    selectedPopUp.style.display = 'none';
    tableActionMenu.style.display = 'none';
    tableCreationMenu.style.display = 'none';

    document.body.style.overflow = '';
}
document.querySelectorAll('.actionMenuAbsoluteButton.exit').forEach(button => {
    button.addEventListener('click', closeOptionMenu);
});

```

Funkcja selectTableIconMenu – pokazuje menu wyboru ikony dla tabeli po kliknięciu w odpowiednie pole, listener dla funkcji.

Funkcja selectTableIcon – zmienia wizualnie ikonę oraz wpisuje ją do ukrytego formularza, aby zapisać ją w tabeli poprzez skrypt PHP.

```

function selectTableIconMenu(){
    const iconSelection = document.querySelector('.iconSelection');
    const iconSelectionMenu = document.querySelector('.iconSelectionMenu');
    iconSelection.style.display = 'none';
    iconSelectionMenu.style.display = 'grid';
}
document.querySelector('.iconSelection').addEventListener('click', selectTableIconMenu);

function selectTableIcon(event){
    const iconSelectionMenu = document.querySelector('.iconSelectionMenu');
    const selectedIcon = event.currentTarget;
    const selectedIconElement = selectedIcon.querySelector('.iconOption use');

    const iconSelection = document.querySelector('.iconSelection');
    const iconSelectionElement = document.querySelector('.iconSelection use')

    const selectedIconStyle = selectedIconElement.cloneNode(true);
    iconSelectionElement.replaceWith(selectedIconStyle);

    const selectedIconId = selectedIconElement.getAttribute('value');
    document.getElementById('iconInput').value = selectedIconId;

    iconSelection.style.display = 'flex';
    iconSelectionMenu.style.display = 'none';
    console.log('Selected Icon ID:', selectedIconId);
}

document.querySelectorAll('.iconOption').forEach(option => {
    option.addEventListener('click', selectTableIcon);
});

```

Ten urywek kodu obsługuje przesyłanie metadanych dla tabeli – zawiera walidację danych. Te dane następnie obsługuje skrypt PHP.

```

document.querySelectorAll('.submitting').forEach(button => {
  button.addEventListener('click', (event) => {
    event.preventDefault();

    const tableInput = document.getElementById('tableInput');
    const iconInput = document.getElementById('iconInput');
    const sourceInput = document.getElementById('sourceInput');
    const tableNameInput = document.getElementById('addTableName');

    tableInput.value = tableNameInput.value.trim();
    sourceInput.value = button.value;

    if (tableInput.value.includes(` `)) {
      alert('Nazwa tabeli nie może zawierać symbolu " ` ". Spróbuj ponownie.');
```

Funkcja deleteTable uzupełnia ukryty formularz nazwą tabeli, którą skrypt php będzie usuwał.

```

function deleteTable(button){
  button.preventDefault();
  var databaseTitle = document.getElementById('actionMenuTitle').innerText;
  var deletionInput = document.getElementById('deletionInput');
  deletionInput.value = databaseTitle;
  if(!deletionInput.value){
    alert('Usuwanie nieudane.');
```

modify-table.js

Funkcja deleteRecord pobiera potrzebne dane, a następnie wysyła je poprzez AJAX do skryptu PHP - jeżeli ten wykona się poprawnie, element zostaje usunięty wizualnie.

```
1  function deleteRecord(event) {
2      event.preventDefault();
3
4      const row = event.target.closest('tr');
5      const recordName = row.querySelector('td').innerText;
6
7      const databaseTitle = document.getElementById('databaseTitle').innerText;
8      const tableInput = document.getElementById('tableInput');
9      tableInput.value = databaseTitle;
10
11     const recordNameInput = document.getElementById('recordNameInput');
12     recordNameInput.value = recordName;
13
14     const formData = new FormData();
15     formData.append('record_name', recordName);
16     formData.append('table_name', databaseTitle);
17
18     fetch('../Skrypty/PHP/record_deletion.php', {
19         method: 'POST',
20         body: formData
21     })
22     .then(response => {
23         if (response.ok) {
24             row.remove();
25         } else {
26             alert('Błąd przy usuwaniu rekordu.');
```

Funkcja addRecord podaje dane do formularza oraz przeprowadza walidację, a następnie przesyła go, po czym dodawaniem rekordu zajmuje się skrypt PHP.

```
41 function addRecord(event) {
42     event.preventDefault();
43
44     const databaseTitle = document.getElementById('databaseTitle').innerText;
45     const tableAdditionInput = document.getElementById('tableAdditionInput');
46     tableAdditionInput.value = databaseTitle;
47     var recordName = document.getElementById("name").value;
48     recordName = recordName.trim();
49     if (recordName.includes('.') || recordName.includes(' ') || recordName.includes('`')) {
50         alert('Nazwa tabeli nie może zawierać symboli " ` ", " \' ", ani " \n ". Spróbuj ponownie.');
```


comparison.js

Najwyższy wynik gracza przechowywany jest w localStorage - następnie pobierany stamtąd przy każdej wizycie.

currentScore to zmienna, która przechowuje aktualny wynik gracza podczas rozgrywki.

Funkcja setTopScore jest używana przy ustanowieniu nowego rekordu.

```
1 let topScore = parseInt(localStorage.getItem("topScore"), 10);
2 var currentScore = 0;
3 console.log(topScore);
4 if (isNaN(topScore)) {
5   |   topScore = 0;
6 }
7 function setTopScore(score) {
8   |   localStorage.setItem("topScore", score);
9 }
```

Ten urywek kodu odpowiedzialny jest za dynamiczne powiększanie elementów podczas poruszania myszką.

```
document.addEventListener("mousemove", (event) => {
  const selectedPopUp = document.querySelector(".selectedPopUp");
  if (selectedPopUp && selectedPopUp.style.display === 'flex') {
    return;
  }
  const mainGameLayout = document.querySelector(".mainGameLayout");
  const leftOption = document.querySelector(".leftOption");
  const rightOption = document.querySelector(".rightOption");
  const leftText = leftOption.querySelector("p");
  const rightText = rightOption.querySelector("p");

  const mouseX = event.clientX / window.innerWidth;

  const rightWidth = 50 + (mouseX - 0.5) * 20;
  const leftWidth = 100 - rightWidth;

  leftOption.style.flex = leftWidth;
  rightOption.style.flex = rightWidth;

  const baseSize = 1;
  const maxIncrease = 1;

  const leftTextSize = baseSize + maxIncrease * (1 - mouseX);
  const rightTextSize = baseSize + maxIncrease * mouseX;

  leftText.style.fontSize = `${leftTextSize}rem`;
  rightText.style.fontSize = `${rightTextSize}rem`;
});
```

Asynchroniczna funkcja fetchNewOptions pobiera nowe opcje po wyborze za pomocą skryptu PHP, a następnie wstawia ich dane w odpowiednich miejscach.

```
39 async function fetchNewOptions() {
40   try {
41     const tableName = new URLSearchParams(window.location.search).get('table_name');
42     const response = await fetch('../Skrypty/PHP/random_records.php?table_name=${tableName}');
43     const data = await response.json();
44
45     if (data.error) {
46       console.error(data.error);
47       return;
48     }
49
50     const leftOptionElement = document.querySelector('.leftOption');
51     const rightOptionElement = document.querySelector('.rightOption');
52
53     leftOptionElement.querySelector('p').textContent = data.left.name;
54     rightOptionElement.querySelector('p').textContent = data.right.name;
55
56     leftOptionElement.dataset.value = parseFloat(data.left.data);
57     rightOptionElement.dataset.value = parseFloat(data.right.data);
58
59   } catch (error) {
60     console.error("Błąd podczas pobierania nowych opcji:", error);
61   }
62 }
```

Funkcja gameOver wyświetla ekran końcowy rozgrywki, uzupełnia go odpowiednimi danymi, oraz aktualizuje najwyższy wynik jeżeli warunek zostanie spełniony.

```
const newRecordMessage = document.createElement('h3');
newRecordMessage.textContent = 'Nowy rekord!';
function gameOver(){
    document.querySelector(".selectedPopUp").style.display = 'flex';
    if(currentScore>topScore){
        setTopScore(currentScore);
        document.querySelector('.scoreInfo').appendChild(newRecordMessage);
        console.log("Nowy najwyższy wynik: " + currentScore);
        topScore = currentScore;
    }
    document.getElementById("currentScore").textContent = "Wynik: " + currentScore;
    document.getElementById("topScore").textContent = "Najlepszy wynik: " + topScore;
}
```

Funkcje dla przycisków “Zagraj ponownie” oraz “Wyjdź”, funkcja dla przycisku wyjścia podczas rozgrywki

```
78 function playAgain(){
79     document.querySelector(".selectedPopUp").style.display = 'none';
80     newRecordMessage.remove();
81     fetchNewOptions();
82 }
83
84 function exit(){
85     window.location.href = "../index/index.php"
86 }
87
88 function returnButton(){
89     if(currentScore==0){
90         window.location.href = "../index/index.php"
91     }
92     else{
93         gameOver()
94     }
95 }
```

Funkcja handleChoice ustala, która opcja została wybrana, odczytuje wartości dla obu, ustala czy wybrano poprawną opcję, wykonuje odpowiednie działania dla obu przypadków.

```
function handleChoice(selectedOption) {
    const otherOption = selectedOption.classList.contains('leftOption')
        ? document.querySelector('.rightOption')
        : document.querySelector('.leftOption');

    const selectedValue = parseFloat(selectedOption.dataset.value);
    const otherValue = parseFloat(otherOption.dataset.value);

    if (selectedValue > otherValue || selectedValue == otherValue) {
        console.log("Poprawny wybór!");
        currentScore++;
        document.querySelector('.currentScore p').textContent = currentScore;
        fetchNewOptions();
    }
    else {
        console.log("Niepoprawny wybór.");
        gameOver();
        currentScore = 0;
        document.querySelector('.currentScore p').textContent = currentScore;
    }
}
```

Back-end

Wykorzystywane są skrypty PHP, polecenia SQL do bazy MySQL.

PHP

Każdy skrypt .php wykonuje jedno, konkretne zadanie. Podstrony z końcówką .php wykorzystują PHP do pobierania nazw i rekordów.

config.php

Skrypt odpowiedzialny za łączenie z bazą i tworzenie jej w razie, gdy jej nie ma.

```
<?php
$server = "localhost";
$username = "root";
$password = "";
$database = "tablix_vs";
$table = "metadata";

$connection = mysqli_connect($server, $username, $password);

if(!$connection){
    die("Połączenie nieudane: " . mysqli_connect_error());
}

$db_query = "CREATE DATABASE IF NOT EXISTS `{$database}`";
if(!mysqli_query($connection, $db_query)){
    die("Tworzenie bazy danych nieudane: " . mysqli_error($connection));
}

mysqli_close($connection);

$connection = mysqli_connect($server, $username, $password, $database);

$table_query = "CREATE TABLE IF NOT EXISTS `{$table}` (
    id INT(11) AUTO_INCREMENT PRIMARY KEY,
    table_name VARCHAR(50) NOT NULL,
    icon_id VARCHAR(100) NOT NULL,
    source ENUM('Custom', 'Spotify') NOT NULL
);";
if(!mysqli_query($connection, $table_query)){
    die("Tworzenie tabeli nieudane: " . mysqli_error($connection));
}
?>
```

metadata_form.php

Skrypt odpowiedzialny za dodawanie danych do tabeli metadata a także tworzenie nowej tabeli.

```

1  <?php
2  require 'config.php';
3
4  $icon_id = $_POST['icon_id'] ?? '';
5  $table_name = $_POST['table_name'] ?? '';
6  $source = $_POST['source'] ?? '';
7
8
9  mysqli_begin_transaction($connection);
10
11  try {
12      $check_metadata_query = "SELECT id FROM metadata WHERE table_name = ?";
13      $stmt = $connection->prepare($check_metadata_query);
14      $stmt->bind_param("s", $table_name);
15      $stmt->execute();
16      $metadata_result = $stmt->get_result();
17
18      if ($metadata_result->num_rows > 0) {
19          header("Location: ../../index/index.php");
20          exit();
21      }
22
23      $query = "INSERT INTO metadata (table_name, icon_id, source) VALUES (?, ?, ?)";
24      $statement = $connection->prepare($query);
25      $statement->bind_param("sss", $table_name, $icon_id, $source);
26
27      if (!$statement->execute()) {
28          throw new Exception("Błąd: " . mysqli_error($connection));
29      }
30
31      $metadata_id = mysqli_insert_id($connection);
32
33      $create_table_query = "CREATE TABLE `$table_name` (
34          id INT AUTO_INCREMENT PRIMARY KEY,
35          metadata_id INT NOT NULL,
36          name VARCHAR(100) NOT NULL,
37          data INT NOT NULL,
38          FOREIGN KEY (metadata_id) REFERENCES metadata(id) ON DELETE CASCADE
39      )";
40
41      if (!mysqli_query($connection, $create_table_query)) {
42          throw new Exception("Niepowodzenie przy tworzeniu tabeli: " . mysqli_error($connection));
43      }
44
45      mysqli_commit($connection);
46      echo "Dodano metadane, tabela '$table_name' została stworzona.";
47  } catch (Exception $e) {
48      mysqli_rollback($connection);
49      echo "Transakcja nieudana: " . $e->getMessage();
50  }
51  if (!empty($_SERVER['HTTP_REFERER'])) {
52      if ($source == "Spotify") {
53          $redirectUrl = "../../Podstrony/api-importing.php?table_name=" . $table_name;
54      } else {
55          $redirectUrl = "../../Podstrony/modify-table.php?table_name=" . $table_name;
56      }
57      header("Location: $redirectUrl");
58      exit();
59  } else {
60      echo "Przesłano formularz.";
61  }
62  }
63  ?>

```

random_records.php

Skrypt wykorzystywany podczas rozgrywki przez skrypt comparison.php. Losuje dwa rekordy z wybranej tabeli.

```
1  <?php
2  require 'config.php';
3  if (!$connection) {
4      die("Połączenie nieudane: " . mysqli_connect_error());
5  }
6
7  if (isset($_GET['table_name'])) {
8      $table_name = $_GET['table_name'];
9      $sql = "SELECT * FROM `$table_name` ORDER BY RAND() LIMIT 2";
10     $result = mysqli_query($connection, $sql);
11
12     $options = [];
13     while ($row = mysqli_fetch_array($result)) {
14         $options[] = $row;
15     }
16
17     if (count($options) === 2) {
18         echo json_encode([
19             'left' => $options[0],
20             'right' => $options[1]
21         ]);
22     } else {
23         echo json_encode(['error' => 'Za mało danych']);
24     }
25 }
26
27 mysqli_close($connection);
28 ?>
```

record_deletion.php

Skrypt odpowiedzialny za usuwanie rekordów z wybranej tablicy.

```
<?php
require 'config.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $record_name = $_POST['record_name'] ?? '';
    $table_name = $_POST['table_name'] ?? '';

    if (empty($record_name)) {
        die("Nie podano nazwy rekordu.");
    }

    if (empty($table_name)) {
        die("Nie podano nazwy tablicy. ");
    }

    $delete_query = "DELETE FROM `$table_name` WHERE `name` = '$record_name'";

    if (mysqli_query($connection, $delete_query)) {
        exit();
    } else {
        die("Błąd przy usuwaniu: " . mysqli_error($connection));
    }
}
?>
```

record_form.php

Skrypt dodaje rekordy do wybranej tablicy po wcześniejszej walidacji.

```
<?php
require 'config.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $record_name = $_POST['name'] ?? '';
    $record_value = $_POST['data'] ?? '';
    $table_name = $_POST['table_name'] ?? '';

    if (empty($record_name) || empty($record_value) || empty($table_name)) {
        die("Błąd - brakuje danych");
    }
    $metadata_query = "SELECT id FROM metadata WHERE table_name = '$table_name'";
    $metadata_result = mysqli_query($connection, $metadata_query);

    if ($metadata_result && mysqli_num_rows($metadata_result) > 0) {
        $metadata_row = mysqli_fetch_assoc($metadata_result);
        $metadata_id = $metadata_row['id'];

        $validation = "SELECT * FROM `$table_name` WHERE name = ?";
        $stmt = $connection->prepare($validation);
        $stmt->bind_param("s", $record_name);
        $stmt->execute();
        $records_result = $stmt->get_result();
        if ($records_result->num_rows > 0) {
            header("Location: " . $_SERVER['HTTP_REFERER']);
            exit();
        }
    }
}
```

```
28     $insert_query = "INSERT INTO `$table_name` ('name', 'data', 'metadata_id') VALUES ('$record_name', '$record_value', '$metadata_id')";
29
30     if (mysqli_query($connection, $insert_query)) {
31         header("Location: " . $_SERVER['HTTP_REFERER']);
32         exit();
33     } else {
34         die("Błąd: " . mysqli_error($connection));
35     }
36 } else {
37     die("Niepoprawna nazwa tabeli - nie znaleziono metadanych");
38 }
39 }
40 ?>
```

table_deletion.php

Skrypt wykonujący transakcje, która ma za zadanie usunąć wybraną tabelę

```
<?php
require 'config.php';

$table_name = $_POST['deletion_name'] ?? '';

if (empty($table_name)) {
    die("Wymagana jest nazwa tablicy.");
}
mysqli_begin_transaction($connection);
try {
    $drop_table_query = "DROP TABLE IF EXISTS `$table_name`";
    if (!mysqli_query($connection, $drop_table_query)) {
        throw new Exception("Błąd przy usuwaniu '$table_name': " . mysqli_error($connection));
    }
    $delete_query = "DELETE FROM metadata WHERE table_name = ?";
    $delete_statement = $connection->prepare($delete_query);
    $delete_statement->bind_param("s", $table_name);
    if (!$delete_statement->execute()) {
        throw new Exception("Błąd przy usuwaniu: " . $delete_statement->error);
    }

    mysqli_commit($connection);
    echo "Metadane oraz tablica `$table_name` usunięte.";
} catch (Exception $e) {
    mysqli_rollback($connection);
    echo "Błąd: " . $e->getMessage();
}

if (!empty($_SERVER['HTTP_REFERER'])) {
    header("Location: " . $_SERVER['HTTP_REFERER']);
    exit();
} else {
    echo "Przesłano formularz.";
}
?>
```

api_importing.php

Skrypt pobiera token dostępu Spotify poprzez cURL wykorzystując dane podane w formularzu, następnie pobiera array danych playlisty

```

<?php
require 'config.php';

$table_name = $_POST['table_name'] ?? '';

function getSpotifyAccessToken($clientId, $clientSecret) {
    $ch = curl_init("https://accounts.spotify.com/api/token");
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    curl_setopt($ch, CURLOPT_POST, true);
    curl_setopt($ch, CURLOPT_POSTFIELDS, "grant_type=client_credentials");
    curl_setopt($ch, CURLOPT_HTTPHEADER, [
        "Authorization: Basic " . base64_encode("$clientId:$clientSecret"),
        "Content-Type: application/x-www-form-urlencoded"
    ]);
    $response = json_decode(curl_exec($ch), true);
    curl_close($ch);
    return $response['access_token'] ?? null;
}

function getPlaylistData($playlistId, $accessToken) {
    $ch = curl_init("https://api.spotify.com/v1/playlists/$playlistId/tracks");
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    curl_setopt($ch, CURLOPT_HTTPHEADER, ["Authorization: Bearer $accessToken"]);
    $response = json_decode(curl_exec($ch), true);
    curl_close($ch);
    return $response;
}

```

Pobranie danych z formularza, walidacja URL, komunikaty w razie błędów, zapytanie mające za zadanie znaleźć id tabeli

```

28
29 $playlistUrl = $_POST['playlist_url'] ?? '';
30 $clientId = $_POST['client_id'] ?? '';
31 $clientSecret = $_POST['client_secret'] ?? '';
32
33 if (!$playlistUrl || !preg_match('/playlistV([a-zA-Z0-9]+)/', $playlistUrl, $matches)) {
34     echo "Niepoprawny URL playlisty.";
35     return;
36 }
37
38 $playlistId = $matches[1];
39 $accessToken = getSpotifyAccessToken($clientId, $clientSecret);
40
41 if (!$accessToken) {
42     echo "Autoryzacja nieudana.";
43     return;
44 }
45
46 $data = getPlaylistData($playlistId, $accessToken);
47
48 if (empty($data['items'])) {
49     echo "Nie znaleziono utworów w playliście.";
50     return;
51 }
52
53 $metadata_query = "SELECT id FROM metadata WHERE table_name = ?";
54 $metadata_stmt = $connection->prepare($metadata_query);
55 $metadata_stmt->bind_param("s", $table_name);
56 $metadata_stmt->execute();
57 $metadata_result = $metadata_stmt->get_result();
58

```

Walidacja, pętla foreach wypełniająca tabelę rekordami z zapytania API

```

59
60 if (!$metadata_result || mysqli_num_rows($metadata_result) === 0) {
61     echo "Nie znaleziono ID tabeli.";
62     return;
63 }
64
65 $metadata_row = mysqli_fetch_assoc($metadata_result);
66 $metadata_id = $metadata_row['id'];
67
68 echo "<ul>";
69 foreach ($data['items'] as $item) {
70     if (!isset($item['track']['name'], $item['track']['popularity'], $item['track']['artists'][0]['name'])) {
71         echo "<li>Brak danych utworu</li>";
72         continue;
73     }
74
75     $record_name = $item['track']['name'] . " - " . $item['track']['artists'][0]['name'];
76     $record_value = $item['track']['popularity'];
77     $insert_query = "INSERT INTO `$table_name` (`name`, `data`, `metadata_id`) VALUES (?, ?, ?)";
78     $insert_stmt = $connection->prepare($insert_query);
79     $insert_stmt->bind_param("sii", $record_name, $record_value, $metadata_id);
80     $insert_stmt->execute();
81
82
83
84 }
85 $redirectUrl = "../Podstrony/modify-table.php?table_name=" . $table_name;
86 header("Location: $redirectUrl");
87 echo "</ul>";
88

```